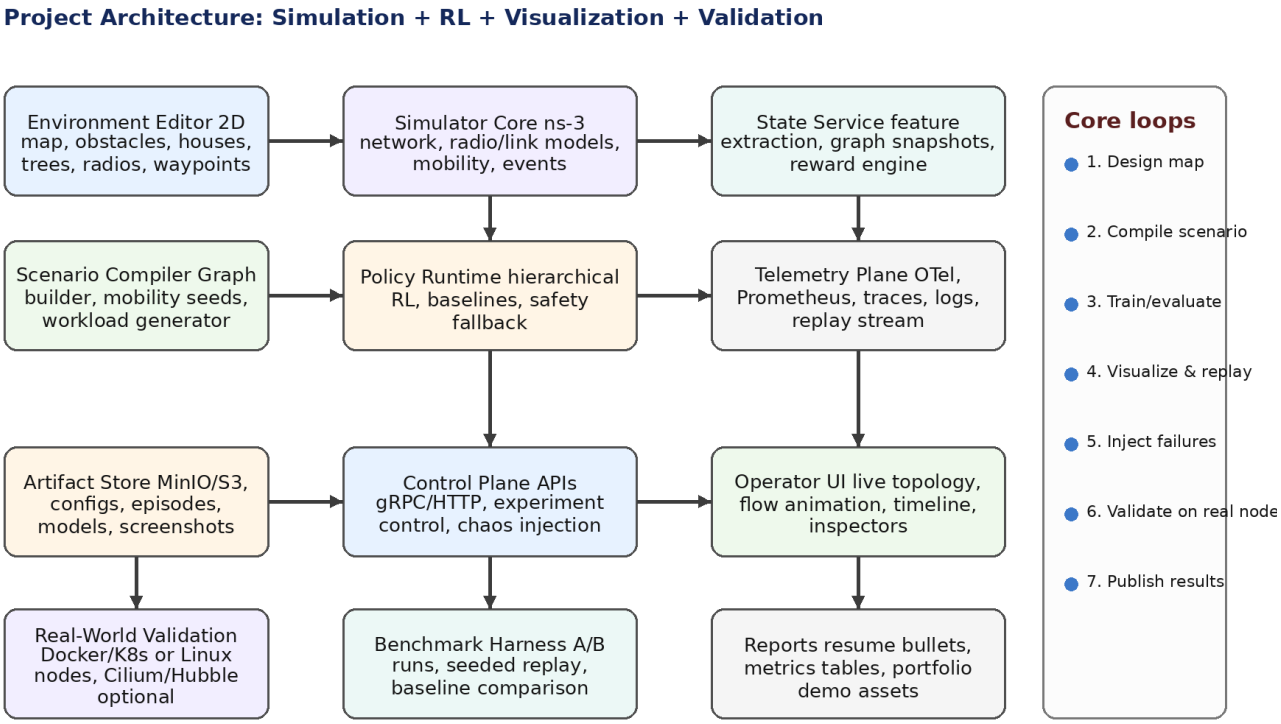
MeshFlight

Self-healing aerial mesh network simulator with hierarchical reinforcement learning, 2D environment authoring, live topology visualization, and real-world validation



System concept: environment editor + simulator + policy runtime + observability + operator console

Document type	Deep project specification
Target outcome	Resume-grade flagship systems + RL project
Recommended build horizon	12-16 weeks for strong V1; 20+ weeks for thesis-grade version

1. Executive Summary

MeshFlight is a research-grade and portfolio-grade platform for designing, simulating, training, visualizing, and validating a self-healing drone mesh network in a physically grounded 2D environment. The system should not only route packets across a changing mesh but should also decide where drones themselves should move in order to preserve coverage, maintain connectivity, reduce interference, and keep important traffic inside latency and reliability targets.

The project is intentionally broader than a toy routing demo. It combines a scenario editor, a network simulator, radio and obstacle models, hierarchical reinforcement learning, reproducible experiment tooling, operator-grade observability, replayable visualization, and a path to real-cluster validation. The result should look credible to systems, networking, infrastructure, robotics, and applied-ML interviewers.

The first release is 2D only. Users lay out walls, houses, trees, interference zones, clients, base stations, and mobile aerial nodes. The simulator turns that authored world into a graph, radio map, workload schedule, and time-varying mobility problem. A hierarchical policy then chooses both movement actions and routing/control actions while a safety layer rejects infeasible decisions.

Primary value proposition: one project that visibly combines systems engineering, RL, networking, observability, simulation, operator UX, and benchmark-driven evaluation.

2. Product Vision and Non-Negotiable Goals

Goal	Specification
What the project must be	A serious platform with distinct subsystems, clear APIs, reproducible experiments, and a polished operator interface. It must be explainable as a system, not as a single model.
What it must not be	Not just shortest-path routing plus a PPO agent. Not just a fixed graph with no physical world. Not just a dashboard that polls a JSON blob. Not just a synthetic benchmark with no failure model.
Target users	You as the builder; interviewers; judges; professors; and future collaborators who want to reproduce scenarios or compare baselines.

Goal	Specification
Core outcomes	A strong GitHub repo, a live demo video, reproducible experiment reports, and 4 to 6 excellent resume bullets with hard metrics.

3. End-to-End User Experience

3.1 Primary workflow

• Author a 2D map: place buildings, trees, walls, no-fly zones, interference sources, client demand zones, and base stations.

• Spawn drone nodes with battery budgets, radio profiles, payload limits, mobility constraints, and initial positions.

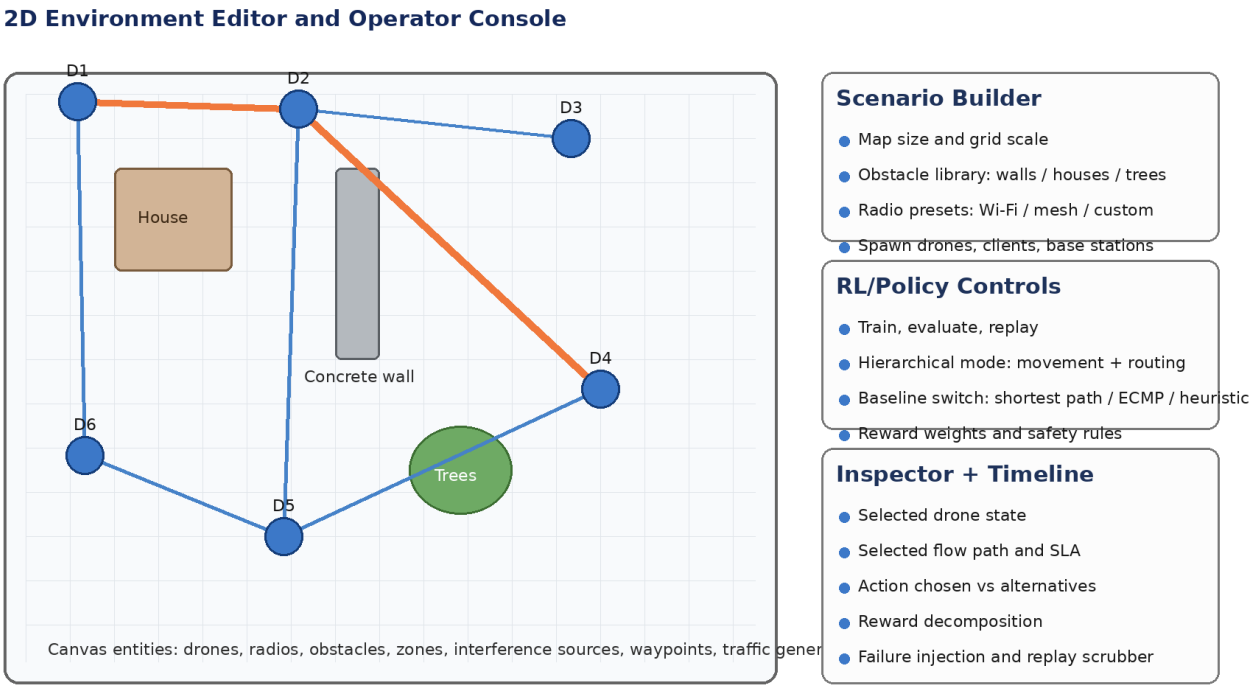
• Configure workload classes such as telemetry, video uplink, safety-critical command traffic, and background synchronization.

• Compile the map into a scenario package: graph snapshots, attenuation hints, movement bounds, event seeds, and evaluation policy.

• Train and evaluate hierarchical RL against strong baselines; replay runs with synchronized timelines and flow inspectors.

• Inject chaos events such as drone loss, moving obstacles, bursty load, and radio degradation; observe self-healing behavior.

• Export artifacts: scenario spec, trained policy, metrics report, screenshots, trace bundle, and benchmark summary.



Concept UI: map editor, live topology, policy controls, and replay/inspection panels

4. System Architecture

Environment Editor

Interactive 2D editor that stores map geometry, obstacle semantics, drone templates, traffic generators, mission phases, and visualization presets.

Scenario Compiler

Transforms authored maps into simulator-ready assets: physical coordinates, obstacle masks, graph seeds, radio parameters, workload schedules, and reproducibility metadata.

Simulation Core

Runs the discrete-event network and mobility simulation. ns-3 is an appropriate backbone because it is explicitly a discrete-event network simulator for Internet systems and publishes release and development documentation covering the tutorial, installation guide, manual, model library, and APIs. [See source appendix]

Policy Runtime

Executes hierarchical RL inference and baseline heuristics. RLlib is a strong candidate because its documentation explicitly supports scalable RL, multi-agent environments, and hierarchical environment patterns. [See source appendix]

State Service

Converts simulator internals into policy-ready graph features, local observations, event deltas, and reward terms.

Telemetry Plane

Instruments simulator, policy, and APIs with traces, metrics, and logs. OpenTelemetry Collector is a good center point because it receives, processes, and exports telemetry through configured pipelines. [See source appendix]

Operator Console

Custom React-based UI for world authoring, live topology animation, flow inspection, failure injection, and replay.

Validation Layer

Runs selected scenarios on real Linux nodes, containers, or Kubernetes so the project can demonstrate partial transfer from simulation to deployment.

5. Environment and World Model

5.1 World coordinate system

The first release uses a bounded 2D plane with metric coordinates, optional snapping grid, and explicit layer ordering. Every entity has position, size or shape, mobility status, and semantic tags.

Map units should be meters. A scenario package stores map bounds, coordinate origin, grid resolution, and unit scale so that replay, rendering, and simulator math remain aligned.

5.2 Obstacle categories

Houses and buildings: opaque or semi-opaque structures with attenuation coefficients, reflection hints, and optional no-fly restrictions.

Walls and fences: linear obstacles that affect line-of-sight, path planning, and packet propagation.

Trees and vegetation: probabilistic attenuation zones that degrade quality without always fully blocking paths.

Dynamic obstacles: moving vehicles or temporary crowd zones introduced after static geometry is stable.

5.3 Node types

Drone relay nodes: battery-constrained aerial mesh nodes with radios, movement limits, and policy-controlled waypoint changes.

Client nodes: handheld devices, sensors, or vehicles that generate traffic but are not themselves decision agents in V1.

Gateway and base nodes: stable uplink nodes connected to a backbone, edge server, or command center.

Interference emitters: fixed or moving sources that perturb channel quality in selected regions.

5.4 Physical realism tiers

Tier 0: abstract graph only for fast debugging.

Tier 1: Euclidean distance plus obstacle penalties.

Tier 2: obstacle-aware line-of-sight and interference zones.

Tier 3: calibrated radio presets informed by real hardware validation runs.

5.5 Environment editor requirements

Area	Must-have capability	Why it matters
Map canvas	Pan, zoom, grid snap, drag-and-drop entities, selection and layering	Makes the project feel like a real tool, not a script-only research environment
Obstacle authoring	Rectangles, polygons, circles, semantic presets for house, wall, tree cluster, no-fly zone	Lets physical layout affect both radio quality and movement planning
Node templates	Drone class presets for speed, battery, radio, altitude band, waypoint step size	Supports reproducible scenario creation and ablation testing
Scenario controls	Event schedule, demand zones, start state, seeds, traffic class mix	Needed for benchmarking and replay reproducibility
Overlay system	Distance rings, estimated signal heat, connected components, bottleneck edges	Improves debugging and makes demos substantially stronger
Persistence	Scenario JSON or YAML plus versioning metadata and screenshot thumbnail	Enables experiment registry and reproducible sharing

6. Drone Mobility and Self-Healing Semantics

In this project, self-healing means more than rerouting packets around a failed edge. The mesh may actively reposition relay drones to restore line-of-sight, reconnect a partitioned region, reduce congestion over a hotspot, or bring a gateway closer to a demand spike.

Each drone therefore participates in two coupled control problems: movement and communication. Movement changes future graph connectivity and radio quality. Communication changes the immediate quality of service seen by flows. This coupling is exactly why the project is strong enough to justify hierarchical RL.

6.1 Movement action space

- Waypoint delta in local coordinates or movement toward one of K sampled candidate waypoints.
- Hold position, return-to-safety, climb or descend band placeholder for later 3D extension, and emergency retreat actions.
- Optional coordinated macro-actions for swarm formation modes such as spread, bridge, escort, or hub clustering.

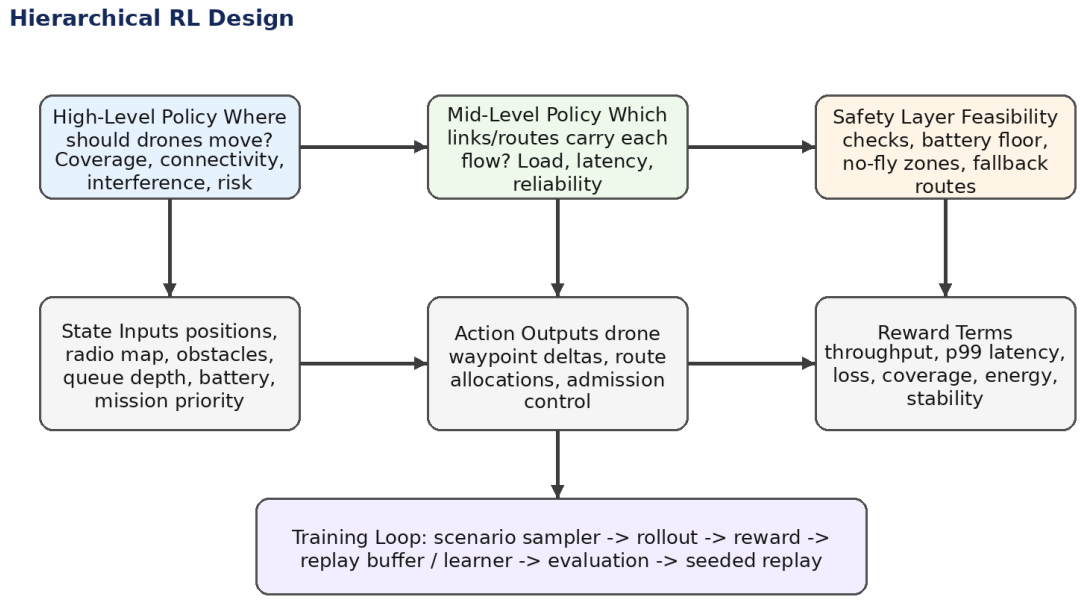
6.2 Movement constraints

- Maximum speed and acceleration per drone class.
- Battery floor below which only safe-return actions are allowed.
- No-fly zones and obstacle collision checks.
- Connectivity safety: optionally forbid a move that would isolate a critical node below a threshold probability.

6.3 What counts as successful self-healing

- Reconnecting clients to a gateway after node loss or obstacle-induced fragmentation.
- Maintaining command-traffic latency below budget even under congestion.
- Restoring multi-hop reachability with minimal movement cost and minimal route churn.
- Gracefully degrading when full restoration is impossible: prioritize mission-critical flows and preserve safe drone state.

7. RL Problem Formulation



Hierarchical design: movement policy + routing policy + safety layer

7.1 Agent structure

Use a hierarchical design instead of a single flat policy. A high-level policy controls drone movement or repositioning, while a mid-level policy controls route allocation, path switching, and admission decisions. The safety layer sits outside learning and rejects clearly unsafe actions. RLlib documentation explicitly discusses both multi-agent environments and hierarchical environment patterns, making it a good fit for this architecture. [See source appendix]

7.2 State representation

- Per-drone local state: position, velocity, battery, neighborhood degree, queue depth, link-quality summary, current load, mission role.
- Global graph summary: connected components, bottleneck scores, gateway reachability, demand heatmap, interference heatmap, top-K congested links.
- Per-flow state: source, destination, class, latency budget, bytes pending, route history, drop count, SLA urgency.
- Obstacle-aware features: line-of-sight flags, estimated attenuation, blocked sector counts, nearest cover or obstacle boundaries.

7.3 Action spaces

- Movement policy: choose waypoint delta or candidate waypoint index for each controllable drone.

- Routing policy: choose next-hop, path template, flow-to-route assignment, queue priority, or drop/defer decision for low-priority traffic.
- Meta-actions: trigger bridge formation, demand-zone rebalancing, or fall back to deterministic heuristic control.

7.4 Reward design

Reward term	Positive direction	Notes
Throughput delivered	Higher	Weighted by flow importance so mission traffic matters more than bulk traffic
Tail latency	Lower	Penalize p95 and p99 breaches rather than only average latency
Packet loss	Lower	Important for demonstrating self-healing under failure
Coverage and connectivity	Higher	Reward number of clients connected to any viable gateway
Energy cost	Lower	Movement should be useful, not constant thrashing
Route churn and instability	Lower	Discourages oscillatory policies that look good only in aggregate

7.5 Training strategy

- Start with routing-only training on fixed topologies to validate state and reward construction.
- Then freeze a reasonable routing policy and train movement policy with routing delegated to heuristic or frozen learned controller.
- Finally run joint or alternating optimization with curriculum learning: small maps, few drones, limited obstacle density, then progressively more difficult scenarios.
- Maintain a seeded replay set of canonical scenarios for regression testing after every major model or simulator change.

8. Baselines and Why They Matter

- Shortest path over static link cost.
- Weighted shortest path with dynamic penalties for loss, utilization, and queue depth.
- ECMP or randomized multi-path for load spreading.
- Greedy movement heuristic: move disconnected drones toward nearest gateway-demand bridge point.
- Rule-based self-healing controller with simple reconnect-first priorities.

A resume-grade result requires beating at least some of these baselines on selected metrics and being honest about where RL does not beat them.

9. Visualization and Operator Experience

Visualization is a first-class subsystem. The portfolio value of the project rises dramatically if the UI makes the system legible: authored map, live topology, active flows, policy choices, and recovery after failure.

Use two layers of visualization. First, a custom simulator interface for world editing, topology animation, and scenario control. Second, industry-style observability dashboards for metrics, traces, and service graphs.

9.1 Main simulator surfaces

Screen	What it shows	Notes
Network overview	2D map, node positions, links, active flows, utilization heat, partitions	Default demo surface
Flow inspector	Selected flow path, SLA budget, reroute history, drops, latency breakdown	Critical for showing self-healing
Policy inspector	Observed state summary, chosen action, alternatives, reward decomposition	Makes RL legible instead of magical
Failure lab	Buttons for node crash, degraded link, interference spike, demand burst, recovery	Lets you stage deterministic demos
Replay compare	Synchronized RL vs baseline	Strong for reports and

Screen	What it shows	Notes
	timeline on same seed	interviews
Ops dashboard	Metrics, traces, node graph, service graph, alerts	Backed by Grafana and telemetry stack

9.2 Rendering and event model

• Use an event stream for low-latency updates: node state changed, link state changed, flow started, path updated, packet dropped, policy action chosen, failure injected, recovery completed, episode ended.

• Also publish periodic snapshots for fast timeline scrubbing and jump-to-time replay.

• Use WebSocket or gRPC streaming between backend and UI. gRPC is well-suited to distributed service communication and works naturally with protocol-buffer API definitions. [See source appendix]

9.3 Recommended frontend stack

• React with either Next.js or Vite for the operator console.

• React Flow or Cytoscape.js for topology editing and interaction; canvas or WebGL overlays for heavy flow animation if needed.

• Grafana for secondary dashboards; its Node Graph visualization is explicitly designed for visualizing relationships between entities and infrastructure maps. Tempo service graphs can represent service relationships when traces are present. [See source appendix]

10. Simulator and Networking Stack

• Simulator core: ns-3 for discrete-event networking and link behavior; custom adapters for map obstacles, drone mobility, and exported event stream.

• Control plane and APIs: Python or Go service layer with gRPC and optional HTTP bridge for the web UI.

• Artifact storage: S3-compatible object store for scenarios, rollouts, models, and rendered assets. MinIO is suitable here because it provides S3-compatible object storage. [See source appendix]

• Containerization: Docker Compose for local development and optional Kubernetes profile for distributed learners or validation cluster.

10.1 Why ns-3 instead of a hand-rolled simulator

Writing a network simulator from scratch would eat time without providing equivalent resume value. ns-3 gives you a documented discrete-event network simulator, model library, and realistic

link behavior, letting you spend time on the novel part: obstacle-aware aerial mesh control, environment authoring, RL, and operator tooling. [See source appendix]

11. Observability, Telemetry, and Replay

- Instrument simulator, policy service, scenario compiler, and UI backend with OpenTelemetry traces, metrics, and logs.
- Use OpenTelemetry Collector in the middle because its configuration model is built around receivers, processors, exporters, and pipelines. [See source appendix]
- Store time-series metrics in Prometheus, which uses a dimensional time-series data model with metric names and labeled dimensions. [See source appendix]
- Use Grafana for dashboards and Tempo for traces. Consider Loki or structured log storage for replay-linked log search.
- If you validate on Kubernetes, Cilium plus Hubble adds cluster-level network observability rooted in eBPF. Hubble documentation describes transparent visibility into communication and networking behavior. [See source appendix]

11.1 Replay linkage

Every experiment run should emit a stable run identifier that ties together scenario file, random seed, policy checkpoint, metrics timeline, trace bundle, and UI replay artifact. This is what turns demos into reproducible engineering.

12. Data Contracts and Storage Layout

- Scenario package: map geometry, node templates, traffic schedule, curriculum tags, simulator config, rendering metadata.
- Episode artifact: action log, reward log, event stream, snapshot sequence, summary metrics, screenshots, checkpoint references.
- Model registry: policy version, training config, scenario curriculum, evaluation bundle, hash of code revision.
- Report bundle: benchmark tables, charts, chosen screenshots, notes, and generated resume bullets.

12.1 Suggested object layout

```
scenarios/<scenario_id>/scenario.json
runs/<run_id>/events.jsonl
runs/<run_id>/snapshots.parquet
runs/<run_id>/metrics.csv
runs/<run_id>/trace_ref.txt
models/<model_id>/checkpoint/
reports/<report_id>/summary.md
```

13. Experiment Plan and Success Metrics

- Static obstacle maps with varying density.
- Demand bursts in one region causing hotspot formation.
- Single-drone and multi-drone failures.
- Interference spikes and partial partition events.
- Coverage tasks where drones must reposition before routing can recover.

13.1 Primary metrics

Metric	Why it matters	Target reporting style
Connected-client ratio	Shows resilience and restoration ability	Report mean plus worst-case during failure window
p95 / p99 latency	Demonstrates quality under stress	Report baseline vs RL deltas
Packet loss	Key for failure scenarios	Break down by traffic class
Movement cost	Prevents gaming by constant repositioning	Report distance traveled and energy proxy
Recovery time	Direct self-healing measure	Time from failure injection to restored service threshold
Route churn	Captures stability	Count path changes per flow and per minute

14. Build Plan

Phase	Definition of done
Phase 0 - framing and contracts	Choose stack, define data contracts, scaffold repo, lock scenario schema, and produce editor wireframes.
Phase 1 - environment editor and compiler	Build 2D authoring tool, save and load scenarios, compile into simulator-ready artifacts.

Phase	Definition of done
Phase 2 - fixed-topology simulator and baselines	Integrate ns-3, basic obstacle penalties, workload generation, strong heuristic routing baselines.
Phase 3 - movement and self-healing semantics	Add drone mobility, failures, coverage metrics, reconnect and bridge baselines.
Phase 4 - RL training stack	Add RLlib environment wrappers, reward engine, curriculum runner, evaluation harness.
Phase 5 - observability and operator UX	Add telemetry stack, replay system, inspectors, and polished dashboards.
Phase 6 - real-world validation	Run selected scenarios on real Linux nodes or containers, compare qualitative behavior and transfer gaps.
Phase 7 - publishing	Prepare benchmark report, screenshots, demo video, portfolio write-up, and resume bullets.

15. Risks and Design Traps

• Too much realism too early. Start with simple obstacle penalties before fully ambitious radio physics.

• Too-weak baselines. Weak baselines make the results look suspicious rather than strong.

• Black-box RL. Without policy inspectors and reward decomposition, the demo will feel hand-wavy.

• Overbuilt UI before simulator correctness. The editor should mature in parallel, but correctness and reproducibility still come first.

• No clear benchmark story. You need fixed seeded scenarios and regression suites from early on.

16. Resume and Portfolio Positioning

• Market the project as a programmable self-healing aerial mesh network platform, not just an RL experiment.

• Lead with the system: environment editor, discrete-event simulator, hierarchical policy runtime, telemetry stack, and operator console.

• Use hard metrics: reduced recovery time, improved connected-client ratio, lower tail latency, lower packet loss, or reduced movement cost versus baselines.

16.1 Example resume bullets to target

• Built a self-healing aerial mesh network simulator with a 2D environment editor, obstacle-aware link modeling, and policy-controlled drone repositioning.

• Designed hierarchical RL policies for coupled movement and routing decisions; benchmarked against shortest-path, ECMP, and rule-based self-healing baselines.

• Implemented operator-grade visualization with live topology, flow replay, failure injection, and decision inspectors backed by traces, metrics, and logs.

• Reduced failure-recovery time by X percent and improved connected-client ratio by Y percent on seeded multi-obstacle scenarios while keeping movement cost bounded.

17. Recommended Technology Stack

Layer	Primary choice	Reason
Simulator	ns-3 + custom adapters	Serious network backbone with existing models and documentation
RL runtime	Python + Ray RLlib	Fast iteration, multi-agent support, scalable training
Backend APIs	Python FastAPI or Go + gRPC	Clear service contracts and streaming support
UI	React + React Flow or Cytoscape.js + canvas overlays	Custom editor and topology interaction
Metrics	Prometheus + Grafana	Industry-standard metrics and dashboards
Traces/logs	OpenTelemetry Collector + Tempo + optional Loki	Structured observability and replay linkage
Artifacts	MinIO	Simple S3-compatible storage for scenarios and run bundles
Validation cluster	Docker Compose first, optional Kubernetes + Cilium/Hubble	Bridges simulation to real deployment and network observability

18. Source Appendix for Current Stack Choices

Technology	Why it appears in this spec	Primary sources
ns-3 documentation	The ns-3 site describes ns-3 as a discrete-event network simulator for Internet systems and provides tutorial, installation, manual, model library, and API documentation.	https://www.nsnam.org/ ; https://www.nsnam.org/documentation/

Technology	Why it appears in this spec	Primary sources
Ray RLlib	RLlib documentation describes scalable reinforcement learning and supports multi-agent and hierarchical environment patterns.	https://docs.ray.io/en/latest/rllib/index.html ; https://docs.ray.io/en/latest/rllib/multi-agent-envs.html ; https://docs.ray.io/en/latest/rllib/hierarchical-envs.html
OpenTelemetry Collector	Collector docs describe receivers, processors, exporters, connectors, and pipelines for telemetry data.	https://opentelemetry.io/docs/collector/ ; https://opentelemetry.io/docs/collector/configuration/ ; https://opentelemetry.io/docs/collector/architecture/
Prometheus	Prometheus documentation describes a dimensional time-series data model with labeled dimensions.	https://prometheus.io/ ; https://prometheus.io/docs/concepts/data_model/
Grafana	Grafana docs describe Node Graph for visualizing relationships between entities and Tempo Service Graph for service relationships.	https://grafana.com/docs/grafana/latest/visualizations/panels-visualizations/visualizations/node-graph/ ; https://grafana.com/docs/grafana/latest/datasources/tempo/service-graph/
gRPC	gRPC documentation introduces it as a high-performance open source RPC framework and explains its protocol-buffer based model.	https://grpc.io/ ; https://grpc.io/docs/what-is-grpc/introduction/
MinIO	MinIO materials and docs describe S3-compatible object storage.	https://www.min.io/ ; https://docs.min.io/enterprise/aistor-object-store/developers/s3-api-compatibility/ ; https://github.com/minio/minio

Technology	Why it appears in this spec	Primary sources
Cilium / Hubble	Cilium and Hubble docs describe distributed network observability and cluster-wide visibility built on eBPF.	https://docs.cilium.io/en/stable/observability/hubble/index.html ; https://docs.cilium.io/en/stable/overview/intro.html